

Advanced Data Management for Data Analysis Assignment 4

Yutao Liu s4213211

November 27, 2024

1 Environment

Programming Language: C

Running System: Linux (via Github Codespace : 2-core • 8GB RAM • 32GB)

2 Introduction

This assignment requires us to explore the impact of branch prediction on performance by implementing two different sorting algorithms.

In order to study the impact of branch prediction, we need to implement the following two sorting algorithms (branched and predicated) in C or C++ programs and compare the performance through experimental results.

3 Experiment

3.1 General Approach of the Algorithm(s)

3.1.1 Branched Sorting Algorithm

- By using the classic Bubble Sort, the branched sorting algorithm could be implemented:
 - Traverse each element of the array and compare adjacent elements pair by pair from left to right.
 - If the element on the left is greater than the element on the right, swap their positions.
 - After each traversal, the maximum value of the current unsorted part will bubble to the end of the array.
 - By gradually reducing the unsorted part, the array is completely ordered.
- Branch misprediction in Bubble Sort algorithm:
 - **if** (**array**[*j*] > **array**[*j* + 1]) is a conditional branch for each comparison.
 - In unordered data, branch prediction is likely to fail, causing the CPU's pipeline to be flushed and reloaded.
 - It can significantly increase the execution time of the sorting algorithm.

- Pseudo-code:

Algorithm 1 Bubble Sort

```

1: for  $i \leftarrow 0$  to  $N - 1$  do
2:   for  $j \leftarrow 0$  to  $N - i - 2$  do
3:     if  $\text{array}[j] > \text{array}[j + 1]$  then
4:       Swap( $\text{array}[j]$ ,  $\text{array}[j + 1]$ )
5:     end if
6:   end for
7: end for

```

3.1.2 Predicated Sorting Algorithm

- Predicated sorting algorithm is used to optimize performance by eliminating conditional branches. Though the implementation also uses the bubble sort structure, it replaces the **if** statement with a condition mask and logical operations to avoid branch prediction problems.
 - Traverse each element of the array and compare adjacent elements pair by pair from left to right.
 - Determine whether to swap by calculating $\text{mask} = \text{if}(\text{array}[j] > \text{array}[j + 1])$
 - Use ternary or bitwise operations to directly update element values without relying on **if** conditions.
- How to avoid the branch misprediction?
 - There are no conditional branches in the **predicated** implementation, and all operations are executed with fixed logic, eliminating the possibility of branch prediction errors.
 - It can significantly increase the execution time of the sorting algorithm.
- Pseudo-code:

Algorithm 2 Predicated Sort

```

1: for  $i \leftarrow 0$  to  $N - 1$  do
2:   for  $j \leftarrow 0$  to  $N - i - 2$  do
3:      $\text{mask} \leftarrow (\text{array}[j] > \text{array}[j + 1])$ 
4:      $\text{temp} \leftarrow \text{array}[j]$ 
5:      $\text{array}[j] \leftarrow \text{mask} ? \text{array}[j + 1] : \text{array}[j]$ 
6:      $\text{array}[j + 1] \leftarrow \text{mask} ? \text{temp} : \text{array}[j + 1]$ 
7:   end for
8: end for

```

3.1.3 Comparison

The following table compares the two different algorithms:

Feature	Branched Sorting	Predicted Sorting
Core Operation	Conditional branching	Conditional masking
Branch Prediction Impact	Heavily influenced by data distribution	Performance remains stable
Time Complexity	$O(N^2)$	$O(N^2)$
Space Complexity	$O(1)$	$O(1)$
Implementation Difficulty	Simple	Moderate

Table 1: Comparison between different sorting algorithms

4 Observation

4.1 Compilation Setting

- As sample data, we use the *Ldiscount-int8.csv* files provided with Assignment 3 (i.e., [0,2147483647]).
- After completing the missing parts of the given program, we can compile and generate the corresponding executable file. In the subsequent compilation, we need to provide two parameters: data volume (Integer, like 10000) and data file path to execute. After execution, the sorting results will be written to a new txt file. In addition, you can also verify the correctness of the program through compilation, or generate performance data logs for subsequent analysis.
- The following table compares the two different algorithms(Data Volume=10000 Integers):

Algorithm	LoadingTime(usec)	SortingTime(usec)	Correctness
Branched In-Place	559	50930	Yes
Branched Out-of-Place	553	3891	Yes
Predicated In-Place	553	62755	Yes
Predicated Out-of-Place	656	8	Yes

Table 2: Comparison between different sorting algorithms

4.2 Performance Analysis

4.2.1 Sorting Time

- *Predicated Out-of-Place* is significantly better than *Branched Out-of-Place*, indicating that eliminating branches improves performance.
- *Branched In-Place* takes longer to sort due to the higher branch misprediction rate.

4.2.2 Impact of Branch Prediction

- When data is out of order, the performance of the branched implementation is significantly degraded.

- The predicated implementation reduces the impact of branch prediction errors on performance by eliminating branches.

4.3 Compilation Notes

4.3.1 Compilation

Run the following command to compile all programs:

- **make**

These executable files will be generated:

- *ADM-2023-A4-sort-branched-inplace*
- *ADM-2023-A4-sort-predicated-inplace*
- *ADM-2023-A4-sort-branched-outofplace*
- *ADM-2023-A4-sort-predicated-outofplace*

4.3.2 Execution

Run the following command to execute all programs:

- **./ADM-2023-A4-sort-branched-inplace 10000 ./data/l_discount-int8.csv**
- **./ADM-2023-A4-sort-predicated-inplace 10000 ./data/l_discount-int8.csv**
- **./ADM-2023-A4-sort-branched-outofplace 10000 ./data/l_discount-int8.csv**
- **./ADM-2023-A4-sort-predicated-outofplace 10000 ./data/l_discount-int8.csv**

Among them, data size(10000) must be integer.If your data files are stored in other folders, please change the file path accordingly. Of course, you can also place them in the data folder according to the given commands.

The sorting results are written to a file (such as output_branched_inplace.txt).

4.3.3 Verification

Run the following command to verify all programs:

- **sort -c ./data/output_branched_inplace.txt**
- **sort -c ./data/output_predicated_inplace.txt**
- **sort -c ./data/output_branched_outofplace.txt**
- **sort -c ./data/output_predicated_outofplace.txt**

If no information is output, the file is sorted correctly.

4.3.4 Record Performance Data

Run the following command to output performance data and redirect it to a log file for subsequent analysis:

```
./ADM-2023-A4-sort-branched-inplace 10000 ./data/l_discount-int8.csv > ./data/output_branched_inplace.txt 2> ./logs/branched_inplace.log

./ADM-2023-A4-sort-predicated-inplace 10000 ./data/l_discount-int8.csv > ./data/output_predicated_inplace.txt 2> ./logs/predicated_inplace.log

./ADM-2023-A4-sort-branched-outofplace 10000 ./data/l_discount-int8.csv > ./data/output_branched_outofplace.txt 2> ./logs/branched_outofplace.log

./ADM-2023-A4-sort-predicated-outofplace 10000 ./data/l_discount-int8.csv > ./data/output_predicate_outofplace.txt 2> ./logs/predicate_outofplace.log
```

5 Conclusion

5.1 Branch Prediction Matters

- The Branched Algorithm relies on if statements for comparisons, leading to frequent branch mispredictions when sorting unordered data, which significantly degrades performance.
- The Predicated Algorithm avoids conditional branches, improving performance by eliminating branch misprediction stalls.

5.2 Performance Comparison

- The Predicated Out-of-Place Algorithm was the fastest, showing that eliminating branches can significantly optimize sorting performance.
- The Branched In-Place Algorithm was the slowest due to frequent branch mispredictions.

In conclusion, this experiment indicates the impact of branch prediction on sorting algorithms and the benefits of using branchless designs.